UNIVERSIDAD TECNOLÓGICA
DE TAMAULIPAS NORTE

Manual de prácticas:
“Sistema de Visión para Robot Colaborativo UR10”

TÉCNICO SUPERIOR UNIVERSITARIO EN MECATRÓNICA

PRESENTA:

Marco Antonio Aguilar
America Lizbeth Favela Agundis
Verónica Marian Martínez Alvarado
Liem Iván Ortiz López

CD. REYNOSA, TAMAULIPAS **ABRIL 2026.**

Repositorio

Se puede acceder al repositorio con la documentación y programas completos mediante el código QR anexado.

O bien, haciendo click en la siguiente liga:
https://github.com/marcoantonioaguilaruttn/Sistema-de-Vision-para-Robot-Colaborativo-UR10/tree/main

Introducción

El presente manual tiene como objetivo guiar al estudiante en la operación del módulo didáctico de visión artificial aplicado a un robot colaborativo UR10 con gripper Robotiq. Las prácticas están organizadas de menor a mayor complejidad, de forma que cada una construye sobre los conocimientos adquiridos en la anterior. Se recomienda seguir el orden propuesto, ya que los subsistemas deben verificarse de forma independiente antes de operar el sistema en su totalidad.

Antes de iniciar cualquier práctica, asegurarse de que el entorno virtual de Python esté activado, que el robot esté encendido y en modo remoto, y que la PC esté conectada a la misma subred que el robot con la dirección IP configurada de forma estática. De igual manera, en la sección inferior se proveen los pasos necesarios para poder usar el módulo de manera correcta. Los pasos también se podrán encontrar en el repositorio adjunto del proyecto.

Preparación

De igual forma especificada en el repositorio, se necesita establecer varias conexiones y configuraciones para poder usar este módulo sin problema alguno.

De igual forma especificada en el repositorio, se necesita establecer varias conexiones y configuraciones para poder usar este módulo sin problema alguno. A continuación se detallan todos los pasos requeridos en el orden correcto.

1.1 Instalar versión de Python mayor a 3.9

Descargar la versión 3.11.9 de Python desde el sitio oficial: https://www.python.org/downloads/release/python-3119/

Durante la instalación, marcar obligatoriamente la opción "Add Python to PATH". Verificar la instalación abriendo una terminal y ejecutando:

python --version

1.2 Instalar Visual Studio Code

Descargar e instalar VS Code desde https://code.visualstudio.com. Una vez instalado, abrir VS Code e instalar la extensión Python de Microsoft siguiendo estos pasos:

1. Abrir VS Code
2. Ir al panel de extensiones (Ctrl+Shift+X)
3. Buscar "Python (Microsoft)"
4. Hacer clic en Install

1.3 Crear el entorno virtual

Abrir una terminal en la carpeta raíz del proyecto y ejecutar el siguiente comando para crear el entorno virtual:

python -m venv .venv

Para activar el entorno virtual, basta con presionar Ctrl + Shift + P en VScode, lo cual mostrará un menu de opciones. Se debe seleccionar la opción *select interpreter*, que mostrará varias opciones, entre ellas el entorno virtual con el nombre venv, se debe seleccionar este.

Nota: El prompt de la terminal debe mostrar (.venv) al inicio de cada línea, lo que confirma que el entorno está activo. Siempre activar el entorno antes de ejecutar cualquier script del proyecto.

1.5 Instalar dependencias

Con el entorno virtual activado, instalar todas las librerías necesarias ejecutando los siguientes comandos uno por uno:

pip install ur_rtde
pip install ultralytics
pip install depthai
pip install opencv-python
pip install numpy

1.6 Estructura del proyecto

La estructura de archivos y carpetas del proyecto debe seguir la siguiente organización para que los scripts se ejecuten correctamente:

```
ProyectoIntegrador_2/
└── scriptsPrueba/
|   ├── pruebaConexion.py
|   └── pruebaGripper.py
|   ├── robotiqGripper.py
|   ├── robotiq_gripper_control.py
|   ├── robotiq_preamble.py
|   └── testCamara.py
├── main.py
├── robotiqGripper.py
├── models/
|   └── best.pt
└── calibracion/
    ├── calibracionCamara.py
    ├── intrinsicosCamara.py
    ├── imagenesDataset/
    └── matriz.npy
```

Nota: El archivo matriz.npy se genera automáticamente al ejecutar calibracionCamara.py. Si aún no existe, consultar la sección de calibración antes de ejecutar el sistema principal.

Práctica 1: Familiarización con la cámara

Objetivo

Que el estudiante se familiarice con la cámara OAK-1 Lite, comprenda el concepto de foco y su efecto sobre la calidad de la imagen, e identifique visualmente las condiciones óptimas de captura para la distancia de trabajo del módulo.

Introducción

La calidad de la imagen capturada por la cámara es un factor determinante en la precisión del sistema de detección de objetos. Una imagen desenfocada introduce ambigüedad en los bordes y texturas de los objetos, lo que reduce directamente la confianza de las detecciones realizadas por el modelo de visión artificial. El parámetro de foco en la cámara OAK-1 Lite se controla de forma manual mediante un valor entero entre 0 y 255, donde valores más altos corresponden a distancias focales más cortas.

Procedimiento

Conectar la cámara OAK-1 Lite a la PC mediante el cable USB y ejecutar el programa de prueba desde la terminal con el entorno virtual activado:

python testCamara.py

Se abrirá una ventana de visualización con el feed en vivo de la cámara. Utilizando los controles del teclado descritos a continuación, ajustar el valor de foco hasta obtener la imagen más nítida posible sobre la superficie de trabajo del módulo:

Tecla +: aumenta el valor de foco en 5 unidades.
Tecla -: reduce el valor de foco en 5 unidades.
Tecla *F*: aumenta el valor de foco en 1 unidad (ajuste fino).
Tecla *D*: reduce el valor de foco en 1 unidad (ajuste fino).
Tecla *Q*: finaliza el programa y muestra el valor de foco final en la terminal.

Anotar el valor de foco que produce la imagen más nítida, ya que este valor deberá utilizarse en prácticas posteriores que involucren la captura de imágenes con esta misma configuración de distancia.

Preguntas de análisis

- ¿Por qué una imagen desenfocada afecta la precisión de la detección de objetos?

- ¿Qué ocurre con el valor de foco óptimo si se modifica la altura de la cámara respecto a la mesa de trabajo?

Práctica 2 — Parámetros intrínsecos de la cámara

Objetivo

Que el estudiante obtenga y analice los parámetros intrínsecos de la cámara OAK-1 Lite, comprenda qué representa cada valor geométricamente y relacione la matriz de la cámara con el modelo de proyección pinhole.

Introducción

Los parámetros intrínsecos de una cámara describen las propiedades ópticas internas del dispositivo, independientemente de su posición en el espacio. Estos parámetros se agrupan en la matriz intrínseca de la cámara, la cual contiene las distancias focales expresadas en píxeles en los ejes horizontal y vertical, denotadas como
fx y *fy*, y las coordenadas del punto principal *cx* y *cy*, que representan la proyección del centro óptico sobre el plano del sensor. Adicionalmente, los coeficientes de distorsión describen las deformaciones radiales y tangenciales introducidas por la lente.

En la cámara OAK-1 Lite, estos parámetros se encuentran almacenados en la memoria interna del dispositivo y pueden leerse directamente mediante la librería DepthAI sin necesidad de un proceso de calibración externo.

Procedimiento

Con la cámara conectada a la PC, ejecutar el archivo de lectura de intrínsecos desde la terminal:

python intrinsicosCamara.py

La terminal mostrará la matriz de la cámara y los coeficientes de distorsión obtenidos para una resolución de 1280×720 píxeles. Registrar los valores obtenidos e identificar en la matriz los elementos correspondientes a
fx, *fy*, *cx* y *cy*.

Como experimento complementario, modificar la resolución especificada en la llamada a
getCameraIntrinsics() dentro del archivo y ejecutarlo nuevamente con una resolución distinta, como 640×480 píxeles. Comparar los valores obtenidos con los anteriores y analizar de qué forma escalan *fx* y *fy* respecto al cambio de resolución, y por qué *cx* y *cy* siempre representan aproximadamente el centro del sensor.

Preguntas de análisis

- ¿Qué relación existe entre *fx* y la distancia focal física de la lente?
- Si la resolución se duplica, ¿en qué factor esperarías que cambiará *fx*?
- ¿Por qué los coeficientes de distorsión no cambian al modificar la resolución?

Práctica 3: Calibración ojo-mano

Objetivo

Que el estudiante ejecute el proceso de calibración ojo-mano para obtener la matriz de transformación rígida que relaciona el sistema de referencia de la cámara con la base del robot, y comprenda la importancia de la cantidad y variedad de poses capturadas en la precisión del resultado.

Introducción

La calibración ojo-mano es el proceso mediante el cual se determina la relación espacial fija entre la cámara y el robot. Esta relación se expresa como una transformación rígida compuesta por una matriz de rotación y un vector de traslación, los cuales permiten convertir cualquier punto expresado en el sistema de referencia de la cámara a coordenadas en el sistema de referencia de la base del robot. Sin esta calibración, el sistema no puede mapear de forma precisa los píxeles detectados en la imagen a posiciones físicas alcanzables por el robot.

El proceso requiere capturar un mínimo de poses del robot con un tablero de calibración de ajedrez visible en la imagen. En cada captura se registran simultáneamente la imagen de la cámara y la pose TCP del robot, que son los dos conjuntos de datos que el algoritmo *calibrateHandEye()* de OpenCV utiliza para resolver la ecuación AX = XB que describe la relación ojo-mano.

Procedimiento

Verificar que el tablero de calibración sea de 6×9 cuadros internos con un tamaño de cuadro de 2.5 cm. Conectar el robot y la cámara, y ejecutar el programa de calibración:

python calibracionCamara.py

El programa solicitará la captura de imágenes de forma interactiva. Presionar la tecla C para tomar cada fotografía, moviendo el robot a una pose distinta entre cada captura. Para obtener una calibración de buena calidad, se recomienda variar tanto la posición como la orientación del robot entre capturas, procurando que el tablero aparezca en distintas regiones de la imagen y con distintas inclinaciones. Capturar un mínimo de 15 poses y un máximo de 30.

Al presionar Enter sin ingresar la tecla C, el programa finaliza la captura e inicia automáticamente el proceso de calibración, generando el archivo *calibracion/matriz.npy* con los resultados.

Como experimento adicional, repetir la calibración con menos de 10 poses y comparar las matrices obtenidas, observando cómo varía el resultado con una muestra reducida y poco diversa.

Preguntas de análisis

- ¿Qué ocurre si todas las poses capturadas son muy similares entre sí?
- ¿Por qué es importante que el tablero aparezca en distintas posiciones y orientaciones dentro de la imagen?
- ¿Qué representan físicamente los valores del vector de traslación obtenido?

Práctica 4: Verificación de la transformación de coordenadas

Objetivo

Que el estudiante verifique manualmente la función de conversión de coordenadas del sistema, conectando el modelo matemático de proyección de cámara con su implementación en código y cuantificando el error de la transformación calibrada.

Introducción

Una vez obtenida la matriz de calibración ojo-mano, el sistema convierte los píxeles detectados en la imagen a coordenadas cartesianas en el espacio del robot mediante un proceso de dos etapas. En la primera, se aplica el modelo de proyección inversa de la cámara para obtener las coordenadas tridimensionales del punto en el sistema de referencia de la cámara, utilizando la distancia focal, el punto principal y la profundidad medida. En la segunda, se aplica la transformación rígida obtenida en la calibración para expresar ese punto en el sistema de referencia de la base del robot.

Este proceso está implementado en la función *convertirCoordenada()* del archivo *main.py*, y su correcta operación es fundamental para que el robot alcance con precisión la posición de cada objeto detectado.

Procedimiento

Colocar un objeto en una posición conocida dentro del campo de visión de la cámara. Ejecutar únicamente la parte de captura y detección del sistema para obtener las coordenadas en píxeles del centro del objeto. Registrar estos valores junto con la profundidad medida manualmente con una regla desde el plano focal de la cámara hasta la superficie de la mesa.

Calcular manualmente la transformación utilizando los valores de la matriz intrínseca obtenida en la práctica 2 y la matriz de calibración del archivo *matriz.npy*, siguiendo las ecuaciones:

$$x_cam = (centroX - cx) / fx * profundidad$$
$$y_cam = (centroY - cy) / fy * profundidad$$
$$P_robot = R_cam2base \ @ \ P_cam + t_cam2base$$

Mover el robot manualmente hasta la posición del objeto y registrar la pose TCP reportada por el Teach Pendant. Comparar la coordenada calculada con la pose real del robot, cuantificando el error en milímetros en cada eje.

Preguntas de análisis

- ¿En qué eje se observa el mayor error?
- ¿A qué fuente de error puede atribuirse?
- ¿Cómo afectaría un error en la medición de la profundidad al resultado final de la transformación?

Práctica 5 — Detección de objetos con el modelo YOLO

Objetivo

Que el estudiante observe el comportamiento del modelo de detección de objetos orientados en tiempo real, analice las detecciones producidas bajo distintas condiciones y desarrolle criterio sobre las limitaciones de los sistemas de visión basados en aprendizaje profundo.

Introducción

El modelo utilizado en este sistema es una instancia de YOLOv8 entrenada para detectar objetos con bounding boxes orientados (OBB, por sus siglas en inglés). A diferencia de los bounding boxes rectangulares alineados con los ejes de la imagen, los OBB incluyen un parámetro de ángulo de rotación que permite describir con mayor precisión la orientación de objetos no alineados con la cámara. Cada detección produce un vector de cinco valores: las coordenadas del centro en píxeles, el ancho y alto del bounding box, y el ángulo de rotación expresado en radianes.

La inferencia se ejecuta en la PC utilizando la librería Ultralytics, que carga el archivo de pesos del modelo y procesa cada frame de la cámara de forma independiente.

Procedimiento

Modificar temporalmente el archivo *main.py* comentando las líneas de inicialización del robot y el gripper al inicio del archivo, de forma que el programa no intente establecer conexión con el hardware robótico. Ejecutar el programa modificado:

python main.py

Observar en la ventana de visualización las detecciones producidas sobre el video en vivo de la cámara. Para cada objeto detectado, la terminal reportará las coordenadas del centro en píxeles y el ángulo del bounding box orientado. Realizar las siguientes pruebas y registrar los resultados:

Colocar una pieza sobre la mesa en distintas orientaciones y observar cómo varía el ángulo reportado. Modificar las condiciones de iluminación cubriendo parcialmente la fuente de luz y registrar el efecto sobre la confianza de las detecciones. Colocar dos piezas del mismo tipo muy próximas entre sí y observar si el modelo las detecta como objetos separados.

Preguntas de análisis

- ¿En qué orientaciones o condiciones el modelo presenta mayor dificultad para detectar correctamente las piezas?

- ¿Qué diferencia existe entre el ángulo reportado por el OBB y la orientación real de la pieza sobre la mesa?

- ¿Cómo podría mejorarse el desempeño del modelo ante las fallas observadas?

Práctica 6: Control básico del robot

Objetivo

Que el estudiante verifique la conexión entre la PC y el robot UR10 mediante la librería ur_rtde, comprenda la diferencia entre movimiento por joints y movimiento cartesiano lineal, y defina de forma segura nuevas posiciones para el robot.

Introducción

La librería ur_rtde implementa el protocolo RTDE (Real-Time Data Exchange) de Universal Robots sobre una conexión TCP/IP, permitiendo el envío de comandos de movimiento y la lectura del estado del robot desde Python. La comunicación requiere que la PC y el robot estén conectados en la misma subred con máscaras de red compatibles.

El robot UR10 admite dos modos principales de movimiento programático. El movimiento por joints, ejecutado mediante
moveJ(), controla directamente los ángulos de cada una de las seis articulaciones del robot y es el más adecuado para alcanzar configuraciones específicas del brazo. El movimiento lineal cartesiano, ejecutado mediante *moveL()*, desplaza el TCP del robot en línea recta en el espacio cartesiano y es el más adecuado para operaciones de pick and place donde la trayectoria del efector final debe ser predecible.

Procedimiento

Asegurarse de que el área de trabajo del robot esté despejada. Verificar que la IP del robot en el archivo *pruebaConexion.py* sea correcta y que el robot esté en modo remoto. Ejecutar el programa de prueba:

python pruebaConexion.py

El robot ejecutará una secuencia de cinco posiciones predefinidas y regresará al origen.

Comparar el movimiento del robot al ejecutar *moveJ()* contra *moveL()* para los mismos puntos de inicio y destino, observando la diferencia en la trayectoria descrita por el TCP en cada caso.

Preguntas de análisis

- ¿En qué situaciones sería peligroso usar *moveL()* en lugar de *moveJ()*?

- ¿Por qué las posiciones se definen en grados en el código pero se convierten a radianes antes de enviarse al robot?

- ¿Qué sucede si se especifica una posición que el robot no puede alcanzar físicamente?

Práctica 7: Control del gripper

Objetivo

Que el estudiante verifique el funcionamiento del gripper Robotiq, experimente con los parámetros de posición, velocidad y fuerza, y determine empíricamente los valores adecuados para el agarre correcto de cada tipo de pieza del módulo.

Introducción

El gripper Robotiq se comunica con la PC a través del robot UR10, utilizando una conexión TCP/IP al puerto 63352 del robot. La librería de control del gripper implementa el protocolo Modbus TCP para el intercambio de comandos y estados con el dispositivo.

El método principal de control es *move_and_wait_for_pos(position, speed, force)*, que acepta tres parámetros enteros en el rango de 0 a 255. El parámetro de posición define la apertura del gripper, donde 0 corresponde a la posición completamente abierta y 255 a la posición completamente cerrada. Los parámetros de velocidad y fuerza controlan la rapidez y la presión del movimiento respectivamente.

Procedimiento

Verificar que el ID del gripper sea 1 en el Teach Pendant del robot, en la sección de instalación de URCaps. Ejecutar el programa de prueba del gripper:

python pruebaGripper.py

El gripper debe cerrarse completamente y luego abrirse, mostrando en la terminal la posición actual en cada paso. Una vez verificado el funcionamiento básico, modificar los parámetros del método *move_and_wait_for_pos()* y experimentar con distintos valores para cada pieza del módulo. Registrar para cada tipo de pieza el valor de posición que permite sujetarla con firmeza sin deformarla, el valor de fuerza mínimo que evita que la pieza se suelte durante el movimiento del robot, y el valor de velocidad que permite un cierre suave sin golpear la pieza.

Preguntas de análisis

- ¿Por qué se realiza una auto-calibración al activar el gripper?
- ¿Qué consecuencias tendría usar una fuerza excesiva sobre piezas de materiales blandos?
- ¿Cómo detectaría el sistema programáticamente si el gripper no logró sujetar el objeto?

Práctica 8: Ejecución del sistema completo

Objetivo

Que el estudiante opere el sistema de pick and place en su totalidad, integrando todos los subsistemas verificados en las prácticas anteriores, y evalúe el desempeño global del sistema cuantificando el error de posicionamiento y las fuentes de fallo observadas.

Introducción

El sistema completo integra cuatro subsistemas que operan de forma coordinada: la cámara OAK-1 Lite captura frames en tiempo real, el modelo YOLOv8-OBB detecta y clasifica las piezas sobre la mesa, la función de conversión de coordenadas transforma los píxeles detectados a poses en el espacio del robot, y el robot UR10 ejecuta la secuencia de movimientos de pick and place para trasladar cada pieza a su posición correspondiente en el módulo.

El programa principal *main.py* gestiona este flujo completo de forma autónoma. Al detectar objetos en la escena, el sistema espera 5 segundos para estabilizar las detecciones y presenta al operador un resumen de los objetos identificados antes de solicitar confirmación para iniciar el ordenamiento.

Procedimiento

Verificar que todos los parámetros del archivo *main.py* estén configurados correctamente: la IP del robot, la ruta al modelo, el punto seguro y el valor de profundidad. Colocar una sola pieza sobre la mesa de trabajo dentro del campo de visión de la cámara y ejecutar el sistema:

python main.py

Observar en la terminal las detecciones reportadas y verificar que la coordenada calculada sea razonable antes de presionar Enter para confirmar el movimiento. El robot debe trasladar la pieza a su posición correspondiente en el módulo. Registrar la posición final de la pieza y medir el error de posicionamiento respecto a la posición destino programada.

Una vez verificado el funcionamiento con una pieza, repetir la prueba con dos y tres piezas de distintas clases simultáneamente en la escena, observando el orden en que el sistema las procesa y registrando cualquier error de clasificación o de posicionamiento que se produzca.

Para finalizar la ejecución del sistema en cualquier momento, presionar la tecla Q sobre la ventana de visualización. El programa cerrará todas las conexiones de forma ordenada.

Preguntas de análisis

- ¿En qué etapa de la cadena de procesamiento se origina el mayor error de posicionamiento observado?

- ¿Qué efecto tiene la orientación de la pieza sobre la precisión del agarre?

- ¿Qué modificaciones al sistema permitirían reducir el error de posicionamiento por debajo de 5 mm?